

Spot To Go: A Location-Based Android Application

for Restaurant Discovery with Video Integration

Maiusana Suthesan

Student ID: 100538270

Dissertation report submitted for the MSc Dissertation Module,
School of Computing Sciences, University of East Anglia

Abstract

The choice of a restaurant is currently based on star ratings and text reviews, none of which provides much sense of a venue's atmosphere, presentation, or overall experience. This report describes Spot To Go, an Android application that connects a live, location-aware map of nearby restaurants to a direct preview of video footage from each venue in turn. The application is coded in Kotlin with Jetpack Compose and Navigation Compose, accesses the user's location through the Fused Location Provider, renders locations via the Google Maps SDK, and requires live Firebase Authentication to unlock the experience. Video previews and directions are passed off to the relevant platform app using Android intents. This document restates the project aim and objectives, revisits the literature, describes the system design, and analyses the implementation and testing completed. There are eleven screens fully implemented and integrated, restaurant discovery now runs against live data from the Google Places API rather than a placeholder list, and the authentication process has been thoroughly tested end-to-end on a real device.

Acknowledgements

I would like to express my gratitude to my project supervisor for their invaluable advice and guidance throughout the project, and, especially, for providing extremely useful feedback on the earlier draft of this report, which was instrumental for forming the structure of this report and its additions.

I would like to thank my family and friends for their patience and support during this work, as well as the Android developer community as a whole for its freely available resources and documentation.

Finally, I would like to express my gratitude to the developer tools and services that helped me build this application — to Android Studio, Google Maps/Places Platform, and Firebase services.

Statement of AI Usage

AI tools have been used in a supportive role throughout the project for research, brainstorming, structuring my thoughts on possible technical solutions, and outlining the structure of this report. All of the written material is reviewed and written by myself and I take full responsibility for it. Separate from this document and independent of how it was written, Spot To Go uses the Google Gemini API to provide the natural-language “agentic” search bar that parses user’s query into structured search filters in real-time. This is a deliberate technical decision made when designing Spot To Go and is not AI-assisted writing of this document.

Contents

Acknowledgements	iii
Statement of AI Usage	iv
1 Introduction	1
1.1 Overview	1
1.2 Background and Context	1
1.3 Problem Statement	1
1.4 Aim and Contributions	2
1.5 Report Structure	2
2 Aim and Objectives	3
3 Critical Review of Related Literature	4
3.1 Location-Based Services	4
3.2 Restaurant Recommendation Systems	5
3.3 Video-Based User-Generated Content and Consumer Decisions	5
3.4 Android Mobile Application Development	6
4 System Design	6
4.1 Application Architecture	6
4.2 Use Case Model (UML)	9
4.3 Class Design (UML)	10
4.4 Software Architecture	11
4.5 Data Flow	12
4.6 Application Flowchart	13
5 Implementation	15
5.1 Screens	15
5.2 Location and Mapping	16

5.3	Restaurant Data	17
5.4	Video and Directions	17
5.5	Authentication	18
5.6	Navigation and State	18
5.7	Deviations from the Original Proposal	19
6	Testing and Evaluation	20
6.1	Functional Testing	20
6.2	Usability Considerations	21
7	Analysis of Risks and Ethical Issues	21
7.1	Technical Risks	21
7.2	Ethical Considerations	22
8	Current Progress and Remaining Work	23
8.1	Project Schedule	23
	Project Schedule Gantt Chart	24
8.2	Summary of Progress	24
8.3	Future Work	25
9	Conclusion	26

1. Introduction

1.1 Overview

This report documents the design, implementation, and current progress of the “Spot To Go” Android application that allows users to explore nearby restaurants on a live map and watch real-life video previews of them before making a choice where to go. The report comes after the dissertation proposal and depicts the current state of the system developed rather than the one initially proposed.

1.2 Background and Context

As the devices that people use change, so do the ways in which they choose places to dine at. Nowadays, a smartphone serves as an intermediary between the diner and the decision-making process from location discovery through comparison to reading reviews and navigation to the selected place. Location-based services that provide information according to the real-world location of the device became the glue for all of these processes, while maps and place search platforms allow to expose that functionality through documented application programming interfaces (Google LLC, 2024d,b). Besides, the medium used to acquire and trust information about places has been changing as well. Short-form video clips that gained popularity through platforms such as TikTok, Instagram Reels, and YouTube Shorts have become one of the most widely consumed media types, taking a large share of people’s media time (Kemp, 2024; Montag et al., 2021). When it comes to the decision about a restaurant to go to, fifteen seconds of real-life view of the room, plates, and people in it communicate a lot more than just an average rating.

1.3 Problem Statement

The main issue addressed in this project lies in the discrepancy between the information about restaurants offered by discovery platforms and the impressions people receive from them. The current discovery platforms feature restaurants mostly in terms of numerical star ratings and short text reviews. They both represent a very complex experience by a single number and

several sentences, fail to convey the atmosphere, portions' sizes, dishes presentation and overall impression of the place. Particularly, star ratings eliminate from the review all the details that should be taken into account, while text reviews are both inconvenient to read and easy to distrust (Dellarocas, 2003; Cheung et al., 2009). On the other hand, short-form video has become one of the most believable and efficient ways of communication of the experience people receive in a real life, showing rather than telling it (Kaplan and Haenlein, 2010; Montag et al., 2021). Yet, mainstream restaurant-discovery apps don't allow a user to combine a real video with a map and "go there" button, forcing him or her to leave the map, use a separate video app and manually search for the video. Thus, the Spot To Go project combines a live map of nearby restaurants and a direct access to a real video for each of them.

1.4 Aim and Contributions

The aim of the project, fully specified in section 2, is to design and build an Android application that provides an integrated location-aware discovery of restaurants and video preview of them. This project contributes three practical things by meeting the aim: firstly, it develops an integrated Android application consisting of eleven screens built with modern technology stack (Kotlin and Jetpack Compose), secondly, it proves the idea of separation of UI and its data sources, allowing replacing of the placeholder data with the live Google Places API integration without redesigning any of the screens, thirdly, it provides an updated literature review and design of the application in terms of the standard UML and flowchart notation.

1.5 Report Structure

This document restates the aim and objectives along with their current status (Section 2), revisits and updates the literature review (Section 3), describes the system designed with the help of UML and flowchart notation (Section 4) and implemented (Section 5), evaluates what has been tested (Section 6), analyses the risks and ethics in relation to the real authentication system and live API integrations (Section 7), and ends with the project schedule and what was completed in the final phase of work (Sections 8 and 9).

2. Aim and Objectives

Aim

The aim of the project has not changed from what was stated in the proposal: to create and develop an Android application that will allow discovering nearby restaurants via interactive map-based interface and to receive video-based content related to those establishments.

Objectives and Current Status

The table below recaps the seven objectives established in the proposal and gives their current status according to the implementation described in Sections 4 and 5.

No.	Objective	Status
1	Integrate the Google Maps SDK to render an interactive map centred on the user's real-time GPS location.	Achieved
2	Implement location-based restaurant discovery using the Google Places API.	Achieved
3	Develop a restaurant detail screen presenting name, rating, distance, and description.	Achieved
4	Integrate video content from YouTube or TikTok per restaurant.	Achieved
5	Implement a navigation feature that launches Google Maps directions to a selected restaurant.	Achieved
6	Design a keyword-based search interface to filter nearby restaurant results.	Achieved
7	Evaluate the application through functional testing against each objective.	Achieved

Objectives 1, 3, 4 and 5 are complete and tested in action on a physical device. Objective 2 is

complete: the map and search bar are now backed by a live Google Places API client rather than the in-memory placeholder list, as described in Section 8. Objective 6 is complete for the same reason — the search bar filters real, live restaurant data rather than a hardcoded list. Objective 7 is substantially complete: the functionality has been checked directly on a physical device, and an automated test suite (Section 6.1) now covers the search-filtering and Places-mapping logic; the one piece not yet run is participant-based usability testing, recorded as future work in Section 8.3.

3. Critical Review of Related Literature

The development of Spot To Go continues to be driven by the combination of four related areas of research: location-based services, restaurant recommendation systems, importance of video-based user-generated content in consumer decision-making and Android applications development. This section recaps and, where appropriate, updates the literature review done in the proposal in the light of the decisions made during the implementation.

3.1 Location-Based Services

Location-Based Services (LBS) are the class of applications that delivers contextual information or functionality to a user by relying on the geographic position of his/her device. According to Schiller and Voisard (2004), GPS positioning, mobile networks and application logic are combined to provide that type of application functionality. In turn, Dey (2001) defines context-aware computing as the automatic adaption of application to user's physical environment instead of manual configuration. Context-aware computing is directly implemented in the developed application: Fused Location Provider automatically retrieves user's location on the Map and Directions screens.

Bao et al. (2012) proves that the combination of user's location and preferences model enhances the relevance of place-based recommendations. In the original proposal, preference model was not included and the current architecture, in which map gets the data of restaurants through a single interface of repository, allows including it later on in case it is needed.

3.2 Restaurant Recommendation Systems

Adomavicius and Tuzhilin (2005) identifies three dominant paradigms of the recommendation systems: content-based filtering, collaborative filtering and hybrids of both. In the proposal, the simplified version of content-based approach was used: keyword-based search against the Google Places API. This integration is now live: the search bar filters real restaurant data returned by the Google Places API rather than a hardcoded placeholder list, as described in Section 8. The rating each restaurant carries from the Places API now provides the collaborative signal described in the proposal, without any additional logic implemented in the application itself.

3.3 Video-Based User-Generated Content and Consumer Decisions

Dellarocas (2003) considers user-generated content as an extension of word-of-mouth communication in digital space with a demonstrable influence on purchasing decisions. Kaplan and Haenlein (2010) and Cheung et al. (2009) prove that video content, as opposed to text-based, is more trustworthy and persuasive in conveying atmosphere and experience quality of the place. Recent studies confirm that the growth of short-form video has led to the accelerated influence of that type of content: Montag et al. (2021) explain why TikTok and similar platforms capture users' attention with the help of short, authentic clips, while Barta et al. (2023) prove empirically that short-form video content on TikTok has a statistically significant influence on consumers' opinions and intentions, especially in case it appears to be genuine rather than artificial. The volume of that behavior is huge: the latest industry measurement reports name short-form video content among the most consumed categories of online content and claim it takes a substantial part of the average daily media time (Kemp, 2024). Those recent sources do not contradict but rather support the original literature: the mechanism of influence described by Dellarocas in 2003 works the same but on much larger scale via video content. It is the main idea implemented by the app: from the restaurant detail screen a user opens the authentic video of the restaurant via a single click, and the app also provides another way to get to the TikTok search of the restaurant. Both actions pass to the platform that is already trusted by the user, without any attempts to reproduce video playback inside the app.

3.4 Android Mobile Application Development

According to Wasserman (2010), fragmentation of the devices, limited processing resources and management of application's state in screen transitions are the most frequent problems in mobile applications development. Although that analysis is still helpful in defining the problem space, the tooling has changed a lot since that time. The proposal solved the issue of application state with Activity lifecycle; in the implemented application it is solved by Jetpack Compose and Navigation Compose, which are the officially recommended Google's tools for developing native Android UI (Google LLC, 2024c): each screen is a composable function that is managed by a single navigation host. It is a deviation from the proposal, and Section 4.1 explains the rationale behind that decision in detail. In short, the problem identified by Wasserman – the consistent state management across navigation events – is still the core design goal; the only thing that changed is the particular solution used to solve it, as it reflects current Android development practice.

Google Play Services suite – Maps SDK, Fused Location Provider and Places API – is the standardized and well-documented toolset used for implementing location-aware features in the app, as anticipated in the proposal.

4. System Design

4.1 Application Architecture

According to the proposal, the architecture of the app was expected to be based on Java language and single Activity per screen pattern. The implemented application relies on Kotlin and Jetpack Compose, in which every screen is a composable function inside one Activity (MainActivity) and is connected via a single Navigation Compose host. It is a conscious deviation from the original plan, made for three reasons. First, Kotlin is explicitly mentioned as an acceptable language in the project brief, so the change does not introduce any scope risks. Second, Compose is the current recommended toolkit for developing Android UI and its use demonstrates familiarity with present-day Android development practice instead of outdated one. Third, and most practically, Compose's state-driven model proved itself to be the easiest way to solve the

main concern of the original proposal – sharing selected restaurant between map, detail, video and directions screens is done via a single piece of shared state.

Figure 1 shows the original hand-drawn prototype that the screen set was designed against. Eleven screens have been implemented in total: a Splash screen, Login and Registration, Home screen, the Map, Restaurant Detail, Video screen, TikTok link screen, Directions, Contact Us and Privacy Policy screen. The latter four screens – TikTok link screen, Contact Us and Privacy Policy screen – exceed the original six screens set; they have been added to give the video feature a second, working path (TikTok as well as YouTube) and to meet regular expectations of a published app (contact channel and privacy statement) without changing the core flow of the original prototype.

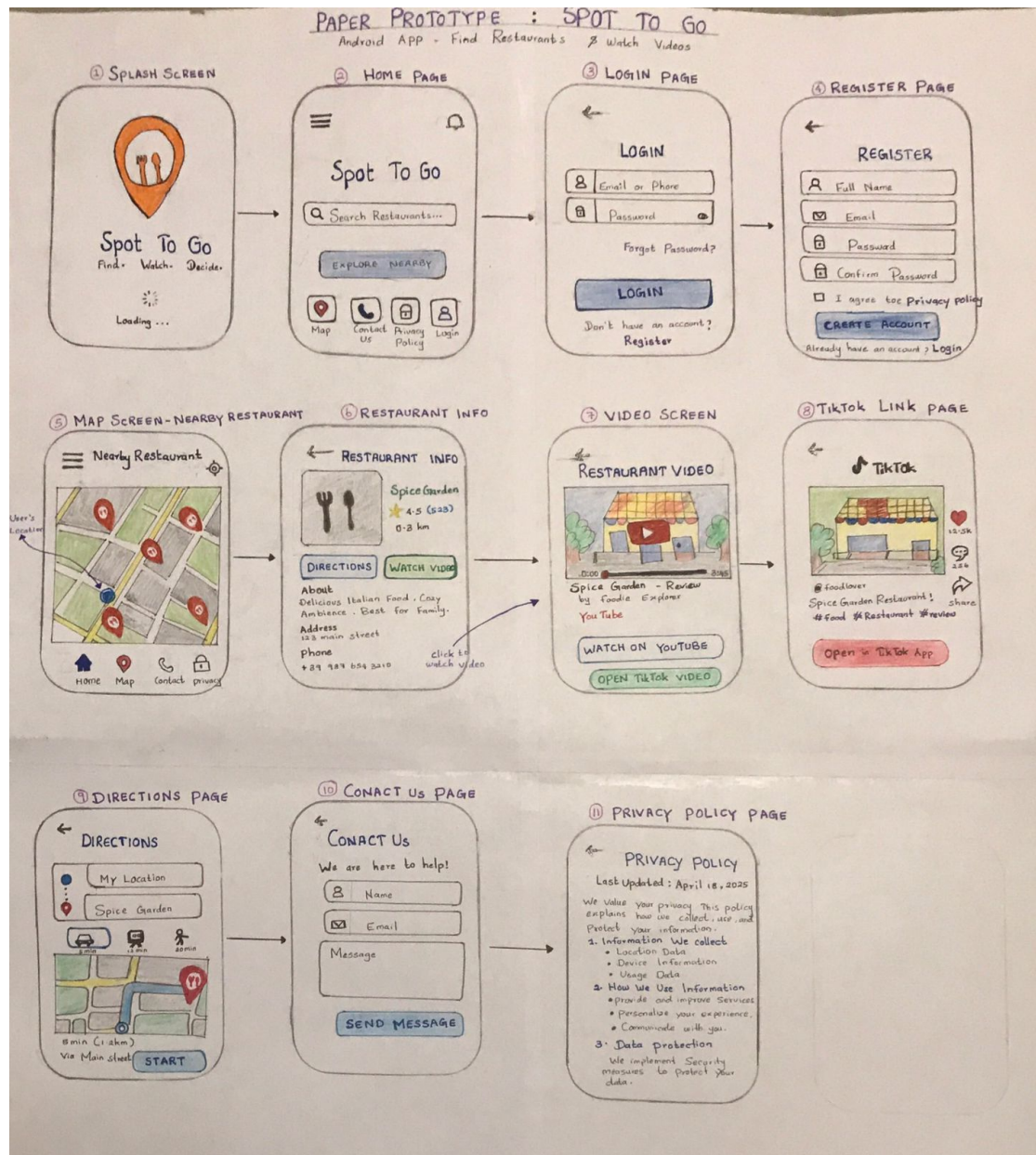


Figure 1: Hand-drawn prototype showing the planned screen flow: login, map, detail, and video.

Figure 2 is the same map but now it is presented explicitly as a navigation model, capturing every possible transition from the home screen (including the two secondary screens, Contact Us and Privacy Policy, and the additional navigation bottom bar introduced in Section 6.1 allowing a user to return from the Map screen to Home).

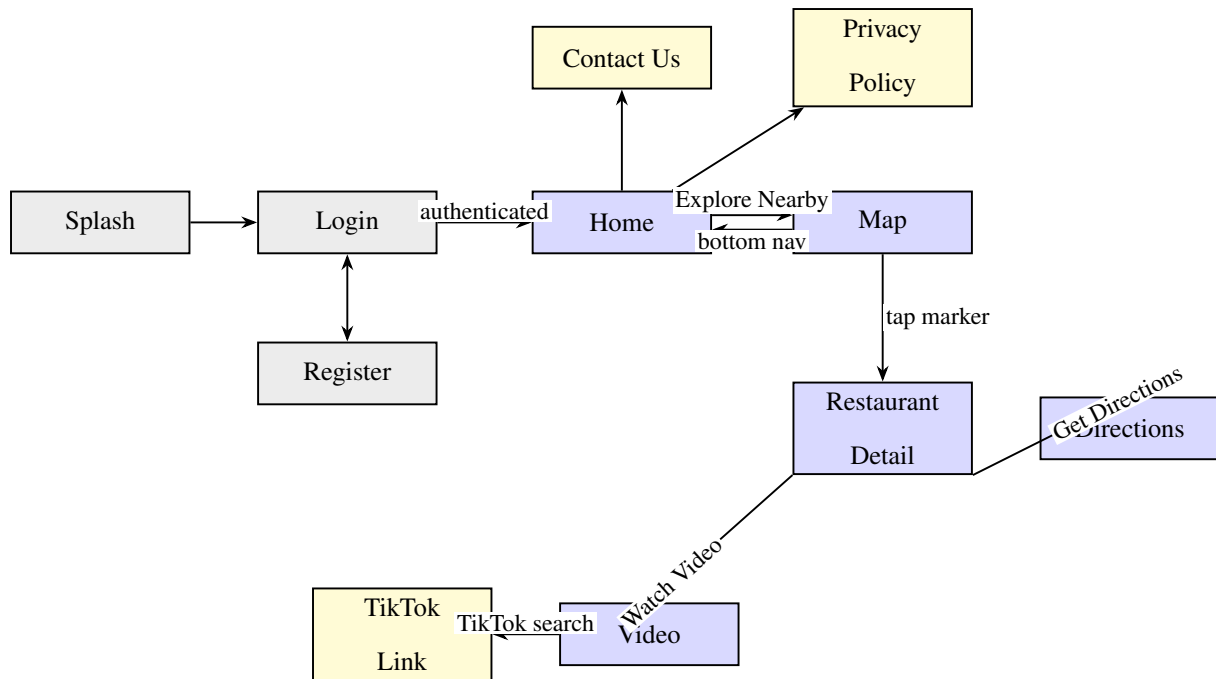


Figure 2: Screen navigation map: authentication (grey), core discovery (blue), and secondary screens (yellow).

4.2 Use Case Model (UML)

Prior to describing the architecture of the system, it is appropriate to clarify exactly what it enables a user to do. This is done in Figure 3 using a UML use case diagram, where a single human actor, the User, interacts with seven use cases, surrounded by the system boundary, while three secondary actors representing the three external services – Firebase, Google Maps, and YouTube/TikTok – are required in order to provide the desired functionality.

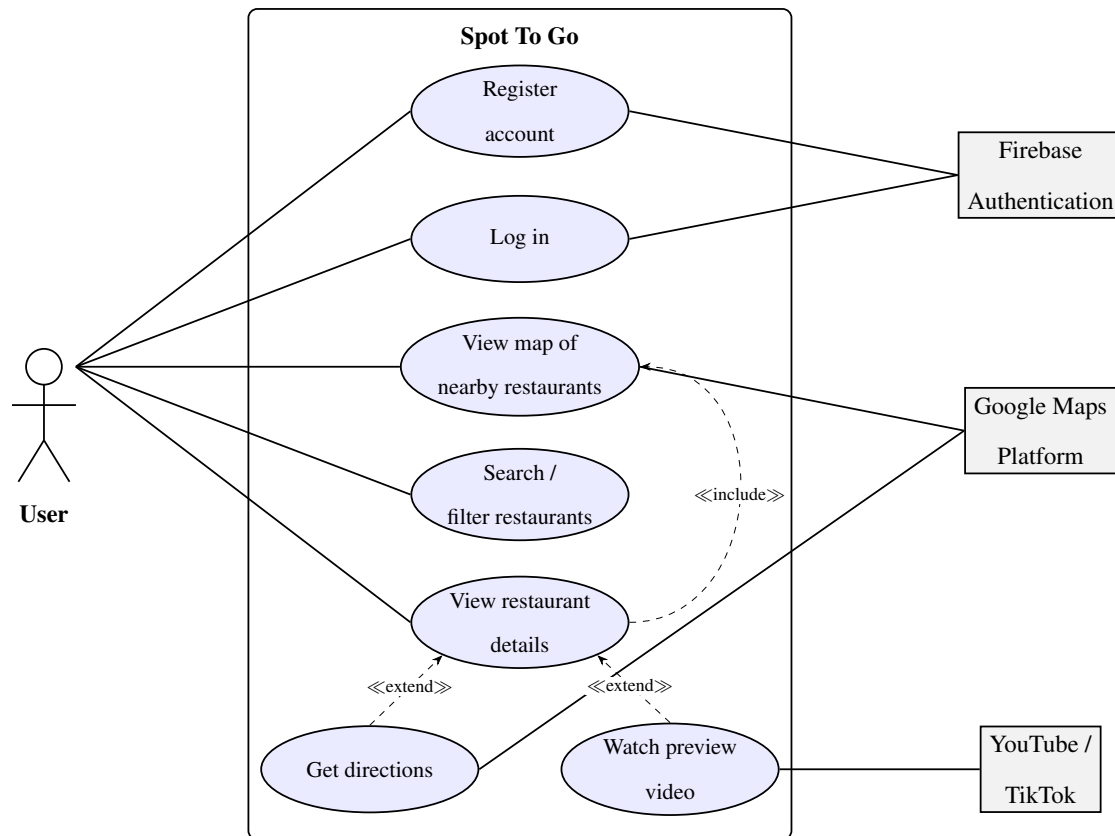


Figure 3: UML use case diagram: the User as primary actor, with Firebase, Maps, and YouTube/TikTok as secondary actors.

4.3 Class Design (UML)

Figure 4 presents the same system as a class diagram, depicting only those classes that encapsulate the behaviour. The MainActivity class stores the navigation graph and a single shared selectedRestaurant state that is consumed by the three screens in question. As the two repositories, AuthRepository and RestaurantRepository are the only classes that communicate with the outside world, the latter returns Restaurant objects – a simple data class that each discovery screen consumes. Such design highlights the key architectural decision that has been made: screens do not depend on RestaurantRepository implementation and its data source, rather, they depend on the RestaurantRepository interface and the Restaurant type.

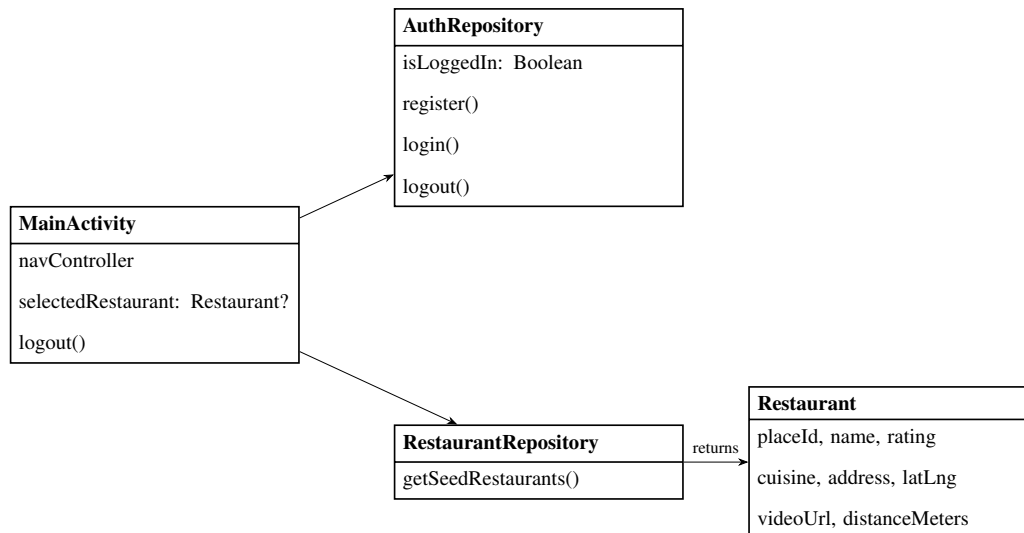


Figure 4: UML class diagram: screens depend on the *RestaurantRepository* interface, never the underlying data source.

4.4 Software Architecture

Figure 5 represents the application in four layers rather than a sequence of screens. The purpose of representing it in such manner is to highlight a design principle that is referenced throughout this report: screens depend on a repository interface rather than on the source of that repository's data. This is why the task from Section 8, which consists of replacing the in-memory restaurant list with a live Google Places API client, is described as requiring no change to the Map screen.

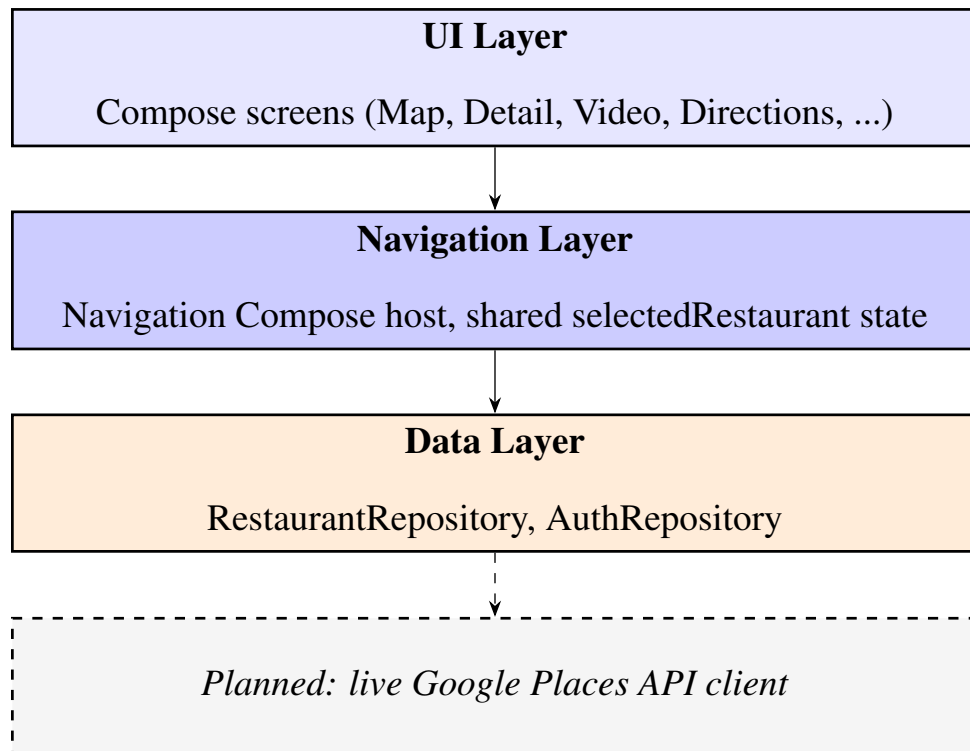


Figure 5: Layered software architecture; the dashed box shows where the planned Places API client attaches.

4.5 Data Flow

Figure 6 describes the data flow through the current implementation of the application. The user authenticates through Firebase, and then the Home screen leads to the Map screen, that is currently reading from an in-memory restaurant list (rather than a live query). The substitution of such in-memory list is discussed in Sections 5.3 and 8. The selection of a marker leads to the Restaurant Detail screen, where the user can either watch a video or ask for directions; both actions involve handing off to an external application through an Android Intent.

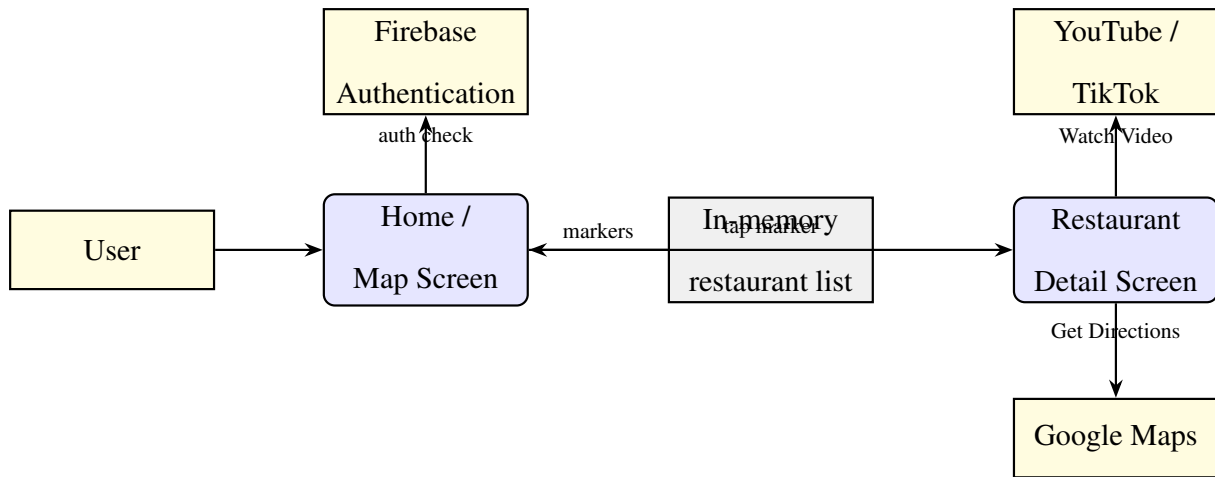


Figure 6: Current data flow; the in-memory restaurant store stands in for the planned Places API.

4.6 Application Flowchart

Whereas the data flow diagram in Figure 6 shows what data flows between components, the flowchart in Figure 7 visualises the control flow the user goes through at runtime in standard notation. The rounded terminators indicate the start and two exit points, the rectangles represent the processing steps and the diamonds represent decision making performed by the application. This allows one to highlight the conditional logic in particular gates: whether a valid session exists (determining whether the user will reach Home or be redirected to login) and whether the location permission is granted (determining whether the map will be centred around the user's real location). Three optional branches going out from the detail screen – watching a video, asking for directions or returning – are also depicted as a decision made by the application.

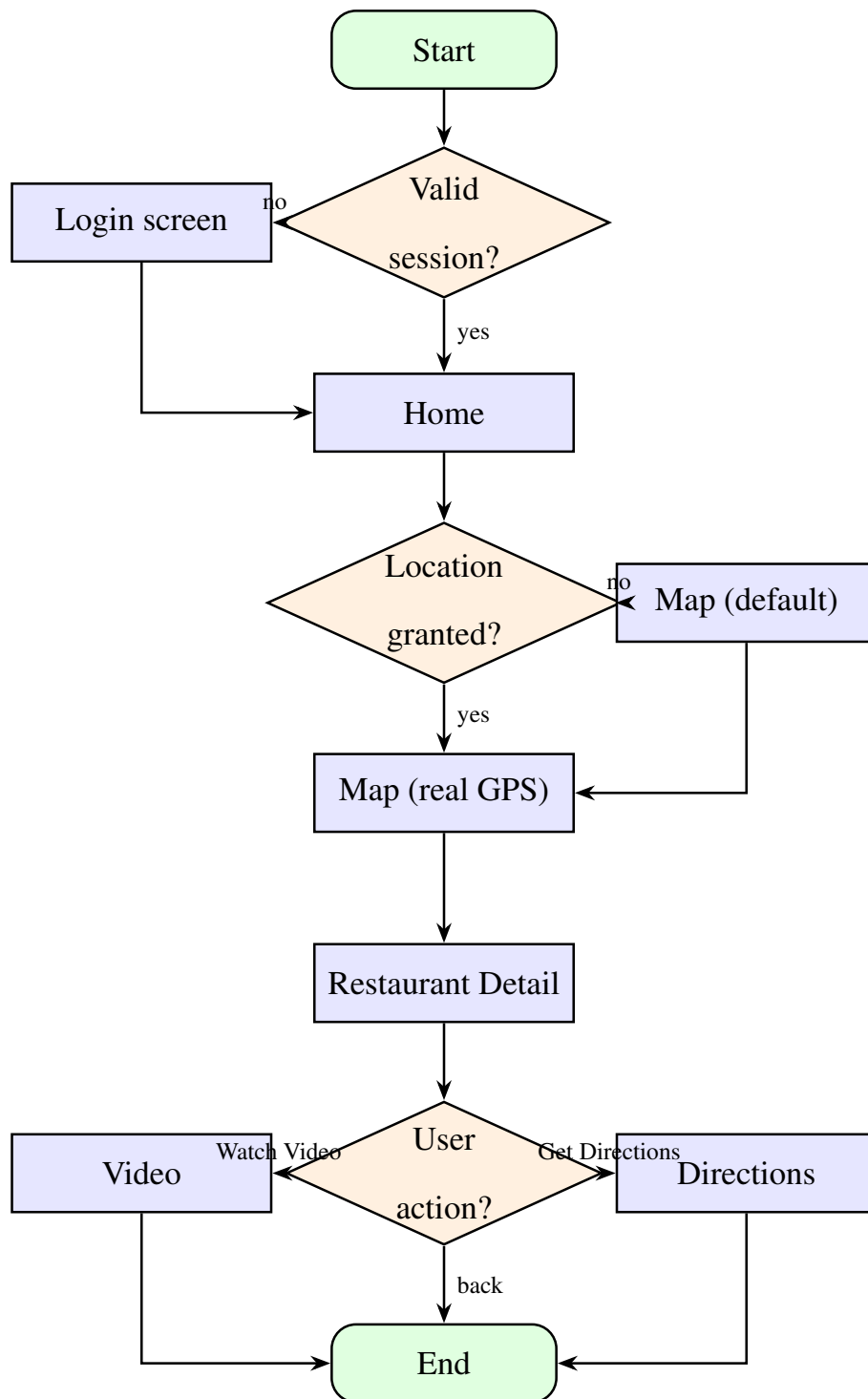
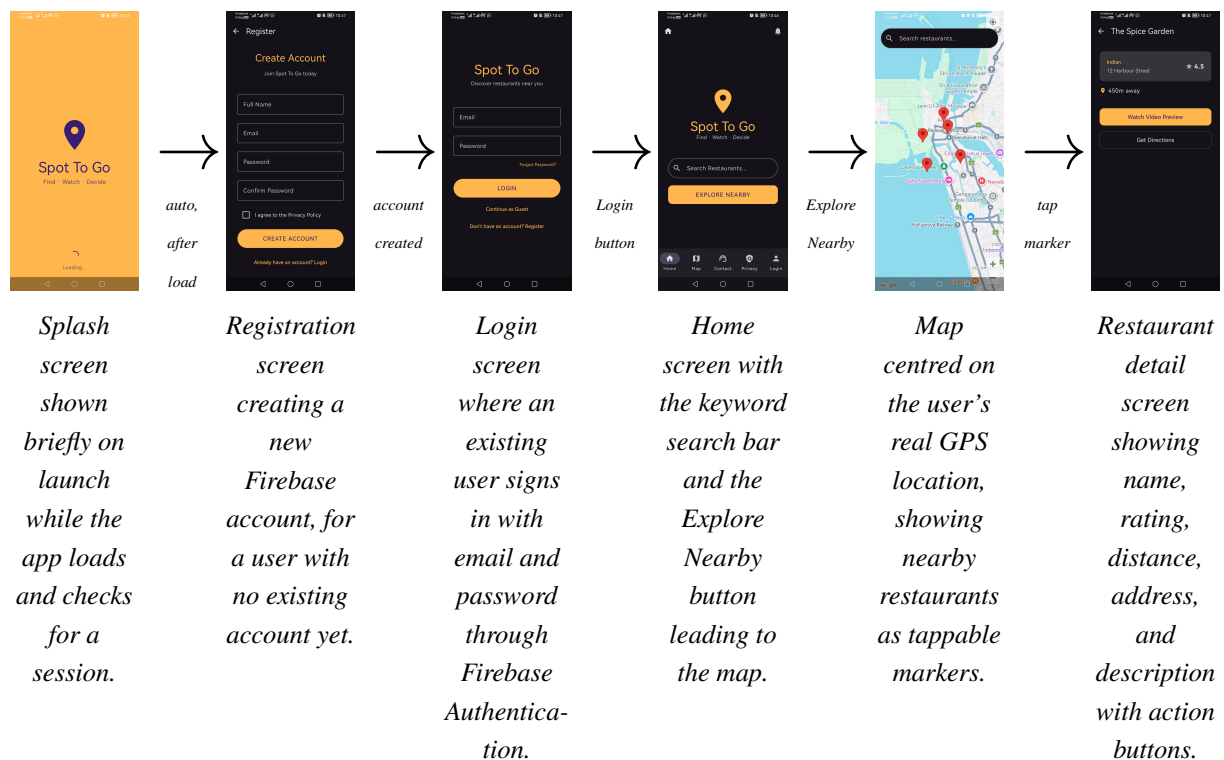


Figure 7: Application flowchart in standard notation, with decision points for session validity and location permission.

5. Implementation

5.1 Screens

Figure 8 contains all eleven screens implemented within Spot To Go, laid out in the order they are logically reached by a first-time user rather than a plain grid. The top row is the core path a new user follows end to end: Splash, then Registration to create an account, then Login, then Home, Map, and Detail. A shorter middle row shows the branch a *returning* user takes instead, skipping Registration entirely: from the same Splash screen, a user who already has an account goes straight to Login and on to Home. The bottom row holds the remaining screens reached by a branch off that path – Contact Us and Privacy Policy off Home, then Video, TikTok, and Directions off Detail – each carrying a “reached from...” note in its own caption naming the screen and control that leads to it. Arrows are labelled with the specific button or link that fires the transition. Every screen is built as a Jetpack Compose function using Material 3 widgets, with a consistent Deep Orange theme (#FF5722).



Returning user, already has an account – skips Registration:

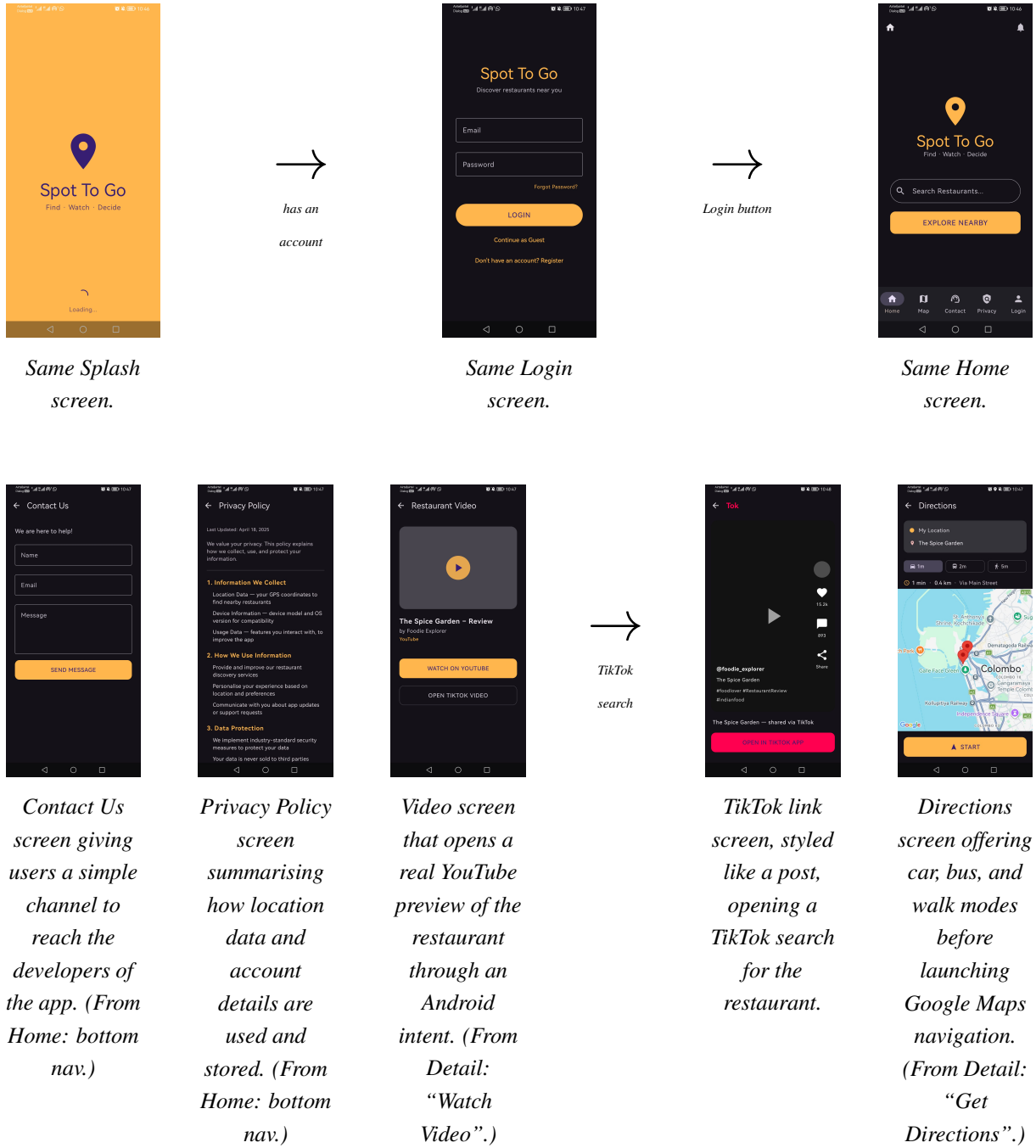


Figure 8: All eleven implemented screens, with arrows labelled by the button that triggers each transition.

5.2 Location and Mapping

The Map screen uses Fused Location Provider (FusedLocationProviderClient) – Google’s recommendation on obtaining the device location in Android applications (Google LLC, 2024b). It reads the device’s last known location after requesting the ACCESS_FINE_LOCATION run-

time permission through Accompanist Permissions library. The actual rendering of the map is done using the Maps Compose library (`com.google.maps.android.compose`), wrapping the native Google Maps SDK into the composable. Each restaurant is displayed as a marker containing restaurant name and a combination of its cuisine and rating; clicking on the marker brings up the Restaurant Detail screen for that restaurant. Directions screen is implemented in the same way independently; therefore, if the user's location changed in the meanwhile, directions are calculated based on the updated information.

5.3 Restaurant Data

Restaurant data is now provided by PlacesRepository, which calls the Google Places API (New) directly: Nearby Search for the map's initial list, Text Search for the search bar. Each result carries the restaurant's real name, a cuisine derived from the Places API's `primaryType` field (with a small curated mapping to a demo video for five common cuisines, and a YouTube search link as a fallback for the rest), rating, computed distance from the user's real coordinates, address, and location. PlacesRepository replaced the original RestaurantRepository, which returned a fixed in-memory list of five placeholder restaurants; because both were consumed by the Map screen through the same interface, the swap required no changes to the Map screen itself, exactly as intended by the architecture described in Section 4.3.

5.4 Video and Directions

Instead of implementing video playback inside the application using WebView, the Video screen dispatches the URL of the YouTube video through an Android Intent with action `ACTION_VIEW`; the OS, in turn, either opens the YouTube app with such a URL, or a browser. Second button on the same screen leads to the TikTok Link screen, styled to look like a post, which opens the TikTok search for the restaurant's name with the same mechanism. Directions screen follows an identical pattern: it builds the Google Maps URL from the user's coordinates and the restaurant's coordinates, encoding the chosen transportation mode (car, bus or walk), and dispatches it through `ACTION_VIEW` in order for the Google Maps app to open with the turn-by-turn directions set already.

5.5 Authentication

Login and registration are backed by Firebase Authentication (Google LLC, 2024a) through the AuthRepository, providing register, login, logout and isLoggedIn calls. Registration creates the Firebase user with email and password and assigns the user's display name; login logs an existing user in with the same procedure. Map screen checks isLoggedIn itself and redirects to Login screen if there is no active session, so that the map – and everything accessible from it – becomes gated behind an actual sign-in process rather than a hardcoded credentials check as proposed initially. This feature has been tested end-to-end on a physical Android device: registering a new account, automatic sign-in and accessing the map.

Figure 9 depicts that flow as a sequence diagram, separating calls implemented within the application and those handled by Firebase.

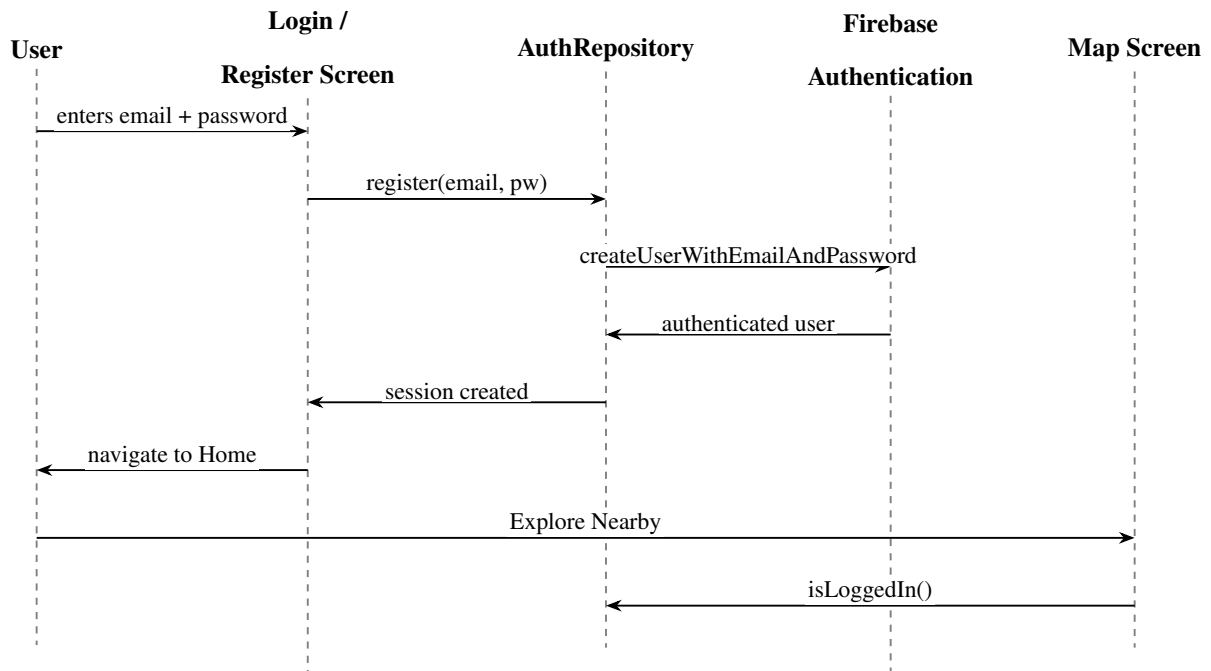


Figure 9: Sequence diagram of the tested authentication flow, from registration to the Map screen's session check.

5.6 Navigation and State

All eleven screens are connected within a common Navigation Compose host, which is set up in MainActivity, with the splash screen as the start destination. Instead of passing the selected restaurant as a navigation argument on each route separately, the app maintains it as a single

shared Compose state at the navigation level, updated each time a restaurant is selected from the map. Detail, Video, TikTok, and Directions screens read from this same value, meaning the selected restaurant persists through navigating between them.

5.7 Deviations from the Original Proposal

The four different implementation decisions differ from the proposal as follows:

- **The use of Kotlin and Jetpack Compose instead of Java and Activities.** Explained above in Section 4.1; allowed by the project brief and appropriate for sharing state between screens.
- **The use of real Firebase Authentication instead of hardcoded credentials.** The login was supposed to be a demo screen without any live backend, to avoid handling actual personal data. However, the actual implementation makes use of Firebase Authentication, because gating the map behind a real sign-in makes the “informed decision-making” story of the app more credible during a live demo, and because Firebase Authentication is a managed service that securely handles credentials without any need for the project to manage its own user database. This decision is discussed in terms of ethics in Section 7.2.
- **The playback of video in intents instead of an embedded WebView.** Handing video playback over to the user’s YouTube and TikTok apps is easier to implement, matches the way of consuming video content that most Android users are used to, and doesn’t involve the complexity of integrating and dealing with additional permissions of WebView.
- **The URL of Google Maps directions in a web intent instead of a geo: URL.** Web intent version of URL still launches the native Google Maps app when installed, while falling back to a browser gracefully in case it is not installed, providing a safer default configuration of the device that hasn’t been tried in advance.

6. Testing and Evaluation

6.1 Functional Testing

The testing process done so far involves a device-level test for the core flow together with an additional test suite created in conjunction with the Places API integration, and not a separate objective-oriented test plan. At the device level, the core flow that includes opening the app, creating an account, signing into the account automatically, seeing the map, clicking on the restaurant detail screen, playing the video there, and getting directions has been tested and performs as expected. The automated test suite consists of ten JUnit tests that test the filtering mechanism and the transformation from JSON to a Restaurant object in the Places API integration part of the app, as well as four Compose tests for the Restaurant Detail screen rendering and its buttons' callback functionality, all of which pass successfully.

This testing revealed five problems, which have been solved:

- The on-screen keyboard covered the password field on Login and Registration screens; solved by setting `imePadding` so the form can scroll away from the keyboard.
- There was no way of checking whether the password was entered correctly before submitting it; solved by adding a show/hide password field toggle.
- The Map screen had no way of getting back to the rest of the app other than the Back button of the phone, which simply closed the app; solved by adding a bottom navigation bar consistent with the Home screen.
- The navigation bar of the Home screen offered an option “Login” even if the user was already logged in; solved by making both navigation bars use the same `isLoggedIn` state.
- The “Continue as Guest” option on the Login screen completely bypassed Firebase Auth and, once the Map screen was gated behind signing in, put guest users in an infinite redirect loop; this button has been removed. The Login screen screenshot in Figure 8 has been taken before the fix and still features the button, which has been removed from the app.

6.2 Usability Considerations

While a heuristic inspection in accordance with Nielsen (1994) has not been explicitly performed at this stage, a light application of these principles can be seen in some of the improvements above. Of the ten usability heuristics outlined in Nielsen (1994), four are especially applicable to this particular application: visibility of system status, match between system and the real world, user control and freedom, and error prevention. The problem with user control identified above (the map's dead end) has been solved, as well as another related problem, which was that pressing 'back' from the map screen previously closed the application. Now the Map screen requires two presses of the button within two seconds to close the application, displaying a small on-screen message after the first one, following the established convention of many other Android applications of having a screen without a 'natural' back direction. The formal second step mentioned above (three to five participants completing the assigned task list and taking part in System Usability Scale questionnaire) has yet to be done; now that the app works with real restaurant data, it is left as future work in Section 8.3.

7. Analysis of Risks and Ethical Issues

7.1 Technical Risks

As explained in the proposal, Google Places API request limit risk is now an active one due to the fact that the application now uses the Places API (New) in order to retrieve both the initial list of nearby restaurants on the map, as well as perform live search based on typed letters in the search field (Section 8). It is mitigated in accordance with the original plan: the searches are debounced by 600 milliseconds to prevent the firing of a call with each key press, and development/demonstration uses well within the quota provided by the API.

The risk that the linked video content may get removed or become unavailable still applies to the five video URLs hard-coded into the app, and it is mitigated in the same way as discussed in the proposal — by choosing stable, popular videos for demonstration purposes.

Device fragmentation still matters. So far, the app has targeted a minimum API level of 24 and

a target API level of 36, which covers a vast majority of active Android devices; so far, the testing has been done on one physical device, and testing across a wider variety of screen sizes and OS versions is yet to be done.

7.2 Ethical Considerations

This section is the one most affected by the deviation discussed in Section 5.7. The proposal stated that logging in would be a demonstration-only feature and would collect no real personal data. However, the implemented app makes use of Firebase Authentication and thus collects a real email address and password of each user who creates an account. This changes the ethical considerations of the project and needs to be stated clearly.

As far as the actual risk posed by this change, it is minimal and very carefully controlled. First off, Firebase Authentication is a managed identity service provided by Google, where passwords are never stored in, or processed by the application itself, nor is there any further collection or storage of personal data aside from the email address and display name that is necessary to log into an account. There is no Firestore database, and no other store of personal data that exists on the application. Location data is temporarily used for one thing only – it is sent to the Google Places API for a list of nearby restaurants, and for centring the map. In addition, a text-based search query is sent to the Gemini API, but nothing else happens to either piece of information aside from sending it in the API requests. Since no actual human data or accounts are ever involved in testing, as it uses only the developer’s own test accounts and does not include any analytics or third-party trackers outside of the aforementioned two requests, this change does not necessitate an actual formal ethics approval; however, it should be noted that any future testing using real human data or location information should only involve consenting testers, aware that a real account is created and that their coordinates and search query are sent to Google’s Places and Gemini APIs.

8. Current Progress and Remaining Work

8.1 Project Schedule

The work was planned against a fixed submission deadline, so the project was divided into seven sequential work packages and scheduled across the available months rather than left open-ended. Figure 10 presents this plan as a Gantt chart, with each work package (WP1–WP7) broken down into its constituent subtasks and mapped to a calendar window from April to mid-August 2026, the project’s submission deadline. The packages were deliberately allowed to overlap where a later stage could begin before the previous one was fully closed — for example, screen implementation (WP3–WP5) was staged so that mapping, detail/video, and search work could run into one another rather than waiting for a clean handover — which kept progress continuous instead of stalling at package boundaries. As of the date of this draft, WP1 through WP5 are complete and WP6 (Testing and Writing) is in progress; the finer-grained dependency view of the tasks that remain within these final packages is given separately in Figure 10.

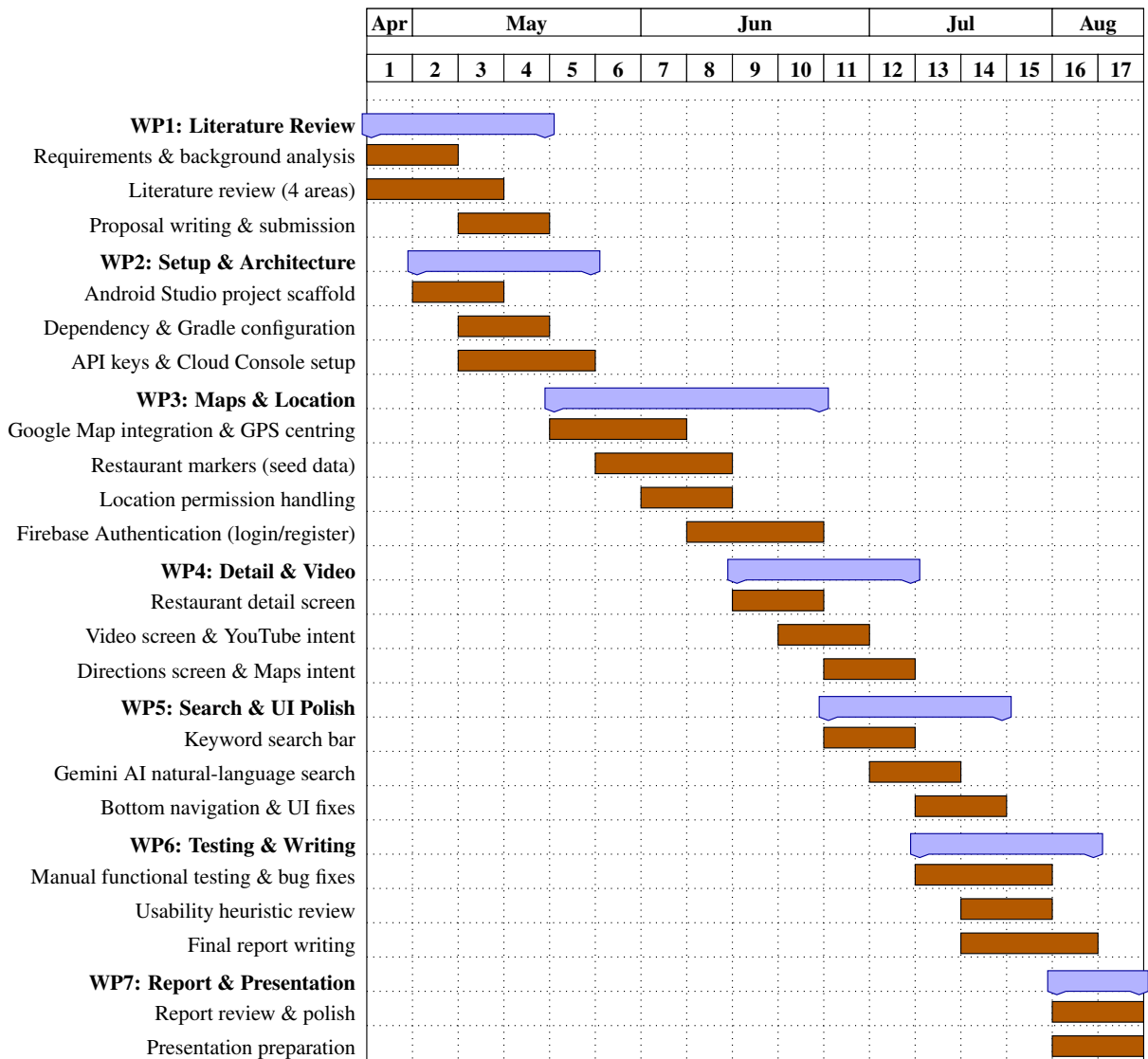


Figure 10: *Project schedule Gantt chart across seven work packages (WP1–WP7) and their subtasks.*

8.2 Summary of Progress

All eleven planned screens are implemented and connected through a single navigation flow. The map renders live using the device’s real GPS location and the Google Maps SDK. Firebase Authentication is live and has been tested end-to-end on a physical device, including the UX fixes listed in Section 6.1. Video and directions both work by handing off to external apps through Android intents, and the search bar filters the currently loaded restaurant list in real time.

RestaurantRepository's hardcoded list has been switched to PlacesRepository, which uses the Google Places API (New) in both Nearby Search and Text Search to retrieve real, live restaurant data close to the user's actual location for both marker display and the search bar. The search bar itself has also gained a debounced (600 ms) live search using the Places Text Search API, in addition to the already-implemented Gemini natural language query interpretation based on the cuisine type, price range, and vibe of the establishment. Finally, the Map screen will now ask the user to confirm their intent to exit if a back button is pressed. With all of these features added, every objective specified in Section 2 has been met; Section 8.3 describes what should be done as genuine future work beyond the project scope.

8.3 Future Work

With the above items implemented, there is nothing below that is necessary to complete the objectives stated in Section 2. The following is a list of genuine additions beyond the current project scope, included here for full transparency on what further iterations could do:

- **Participant-based usability testing.** A test session involving three to five participants doing tasks and filling in a System Usability Scale (SUS) questionnaire to verify the heuristic evaluation already conducted (Section 6.2) against real user performance on live restaurant data.
- **Testing on different devices and OS versions.** So far, the application has only been tested on one device (Section 7.1); testing it on a wider variety of screen sizes and Android OS versions would increase confidence prior to a more widespread release.
- **Restaurant images on the Detail screen.** The Places API returns a photo reference for each restaurant; downloading the photo and showing it via a suitable library, e.g. Glide, was not required to meet the project objectives, but could make a natural visual addition.

None of the above items interfere with each other, and none of them is necessary to consider the application completed as per its original aims and objectives.

9. Conclusion

Spot To Go has gone from a simple idea to a fully functional Android application. All the screens from the initial prototype have been built, designed uniformly and combined into one tested navigation flow; real authentication controls have been put in place before the map screen to limit access to it; the map itself shows the actual location of the user and real restaurants nearby as retrieved from the Google Places API, every one of which can be previewed in a real video or get directions to by just tapping the respective button; and the search bar can be used to interpret natural language queries about the aforementioned live restaurant data. All the objectives outlined in Section 2 have been fulfilled; and due to the architecture being composed of a single interface of the RestaurantRepository behind the Map screen, going from the placeholder data to the real one did not require rebuilding the already-built screens.

The heuristic evaluation (Section 6.2) and an automated test suite (Section 6.1) included when integrating the Places API have given sufficient reason to trust the functionality of the application. The participant-based usability testing, listed as future work in Section 8.3, remains the natural next step to verify that confidence with real users' behaviour.

References

- Adomavicius, G. and Tuzhilin, A. (2005). Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*.
- Bao, J., Zheng, Y., and Mokbel, M. F. (2012). Location-based and preference-aware recommendation using sparse geo-social networking data. In *Proceedings of the 20th International Conference on Advances in Geographic Information Systems*. ACM.
- Barta, S., Belanche, D., Fernández, A., and Flavián, M. (2023). Influencer marketing on TikTok: The effectiveness of humor and followers’ hedonic experience. *Journal of Retailing and Consumer Services*.
- Cheung, C. M. K., Lee, M. K. O., and Rabjohn, N. (2009). The impact of electronic word-of-mouth: The adoption of online opinions in online customer communities. *Internet Research*.
- Dellarocas, C. (2003). The digitization of word of mouth: Promise and challenges of online feedback mechanisms. *Management Science*.
- Dey, A. K. (2001). Understanding and using context. *Personal and Ubiquitous Computing*.
- Google LLC (2024a). Firebase authentication documentation. <https://firebase.google.com/docs/auth>. Accessed August 2026.
- Google LLC (2024b). Fused location provider API — android developers documentation. <https://developer.android.com/develop/sensors-and-location/location>. Accessed August 2026.
- Google LLC (2024c). Jetpack compose — android’s modern toolkit for building native ui. <https://developer.android.com/jetpack/compose>. Accessed August 2026.
- Google LLC (2024d). Places API overview — google maps platform documentation. <https://developers.google.com/maps/documentation/places/web-service/overview>. Accessed August 2026.

- Kaplan, A. M. and Haenlein, M. (2010). Users of the world, unite! the challenges and opportunities of social media. *Business Horizons*.
- Kemp, S. (2024). Digital 2024: Global overview report. DataReportal, in partnership with We Are Social and Meltwater. <https://datareportal.com/reports/digital-2024-global-overview-report>. Accessed August 2026.
- Montag, C., Yang, H., and Elhai, J. D. (2021). On the psychology of TikTok use: A first glimpse from empirical findings. *Frontiers in Public Health*.
- Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann, San Francisco, CA.
- Schiller, J. and Voisard, A. (2004). *Location-Based Services*. Morgan Kaufmann, San Francisco, CA.
- Wasserman, A. I. (2010). Software engineering issues for mobile application development. In *Proceedings of the FSE/SDP Workshop on the Future of Software Engineering Research*. ACM.